

FSAE Force Calculator — MATLAB Tutorial

Built for a Python programmer · everything derived from hardpoints

Design rule: the only numbers you type by hand are physical properties nobody can measure from geometry — mass, friction coefficient, speeds, spring rates. **Every length, angle, lever arm and ratio is computed from the LOTUS hardpoints.**

This document assumes you have not read CAE Report 9.0. Every quantity is defined before it is used.

PART 0 — MATLAB for someone who knows Python

The concepts transfer. The syntax fights you in specific, predictable ways. This section is the map.

0.1 The single biggest gotcha



Python: `a[0]` first element
MATLAB: `a(1)` first element

MATLAB indexes from 1, and uses round brackets for indexing. Round brackets also mean "function call", so `a(1)` is ambiguous to read but unambiguous to MATLAB — it checks whether `a` is a variable or a function.

Second gotcha:



Python: `range(1,5)` → 1, 2, 3, 4 (end exclusive)
MATLAB: `1:5` → 1, 2, 3, 4, 5 (end INCLUSIVE)

A loop that ran 4 times in Python runs 5 times in MATLAB with what looks like the same bounds.

0.2 Translation table — basics

Python / NumPy	MATLAB	Note
import numpy as np	—	Linear algebra is built in
a = np.array([1,2,3])	a = [1 2 3]	Space or comma separates columns
a = np.array([[1],[2],[3]])	a = [1; 2; 3]	Semicolon starts a new row
a.T	a'	Apostrophe transposes
a[0]	a(1)	1-based
a[-1]	a(end)	No negative indexing
a[1:3]	a(2:3)	Inclusive on both ends
A[:,2]	A(:,3)	Colon means "all"
A[1:3, 0:3]	A(2:3, 1:3)	
len(a)	numel(a)	length gives longest dimension only
a.shape	size(a)	
np.zeros((3,3))	zeros(3,3)	No tuple
np.eye(3)	eye(3)	
a ** 2	a ^ 2	
a != b	a ~= b	Tilde, not exclamation
not x	~x	
and / or	&& /	
True / False	true / false	Lowercase
# comment	% comment	

0.3 Translation table — the operator that will bite you

NumPy and MATLAB are opposites here:

Operation	Python / NumPy	MATLAB
Element-wise multiply	A * B	A .* B
Matrix multiply	A @ B	A * B
Element-wise power	A ** 2	A .^ 2
Element-wise divide	A / B	A ./ B

In NumPy * is element-wise. In MATLAB * is matrix multiplication. If you write `v * v` expecting element-wise squaring you'll get a dimension error, or worse, a scalar. The dot prefix means "element by element."

0.4 Translation table — linear algebra

Python / NumPy

MATLAB

<code>np.linalg.norm(v)</code>	<code>norm(v)</code>
<code>np.cross(a, b)</code>	<code>cross(a, b)</code>
<code>np.dot(a, b)</code>	<code>dot(a, b)</code>
<code>np.linalg.solve(A, b)</code>	<code>A \ b</code>
<code>np.linalg.inv(A)</code>	<code>inv(A)</code> — avoid, use <code>\</code>
<code>np.linalg.cond(A)</code>	<code>cond(A)</code>
<code>np.sqrt(x)</code>	<code>sqrt(x)</code>
<code>np.abs(x)</code>	<code>abs(x)</code>
<code>np.degrees(x)</code>	<code>rad2deg(x)</code>
<code>np.radians(x)</code>	<code>deg2rad(x)</code>
<code>np.arccos(x)</code>	<code>acos(x)</code> or <code>acosd(x)</code> for degrees
<code>np.sin(x)</code>	<code>sin(x)</code> or <code>sind(x)</code> for degrees

Use the d-suffixed trig functions. `sind(47)` takes degrees directly. This removes an entire class of bug — you never convert anything.

0.5 Translation table — control flow

Python

MATLAB

<code>for i in range(6):</code>	<code>for i = 1:6 ... end</code>
<code>if x > 0: ... else:</code>	<code>if x > 0 ... else ... end</code>
<code>elif</code>	<code>elseif</code>
indentation defines blocks	<code>end</code> keyword defines blocks
<code>def f(x): return y</code>	<code>function y = f(x) ... end</code>
<code>return a, b</code>	<code>function [a, b] = f(x)</code>
<code>a, b = f(x)</code>	<code>[a, b] = f(x)</code>

MATLAB does not use indentation for structure. Every `for`, `if`, `while` and `function` needs a matching `end`. Indentation is cosmetic (though `Ctrl+I` auto-indents your whole file).

0.6 Dict → struct

Your closest analogue to a Python dict is a **struct**.

Python

MATLAB

<code>d = {}</code>	struct is created on first assignment
<code>d['LBJ'] = [150, 527, 145]</code>	<code>hp.LBJ = [150 527 145]</code>
<code>d['LBJ']</code>	<code>hp.LBJ</code>
<code>d[key]</code> where <code>key</code> is a variable	<code>hp.(key)</code> — dot then parentheses
<code>d.keys()</code>	<code>fieldnames(hp)</code>
<code>'LBJ' in d</code>	<code>isfield(hp, 'LBJ')</code>

Difference from dict: struct field names must be valid identifiers — no spaces, no leading digits.

0.7 List of strings → cell array

MATLAB matrices hold numbers of one type. For a list of strings you need a **cell array**, which uses curly braces:

Python

MATLAB

<code>names = ['a', 'b', 'c']</code>	<code>names = {'a', 'b', 'c'}</code>
<code>names[1]</code>	<code>names{2}</code> — curly braces to extract
<code>len(names)</code>	<code>numel(names)</code>

`names(2)` with round brackets returns a 1×1 *cell containing* 'b'. `names{2}` returns the string 'b' itself. This distinction causes a lot of confusion — when you want the contents, use curly.

0.8 Printing

Python

MATLAB

<code>print(x)</code>	<code>disp(x)</code>
<code>print(f"L = {x:.2f} mm")</code>	<code>fprintf('L = %.2f mm\n', x)</code>

fprintf does not add a newline. You must write `\n` yourself. Format specifiers are C-style, same as Python's old % formatting:

Code	Meaning
<code>%8.2f</code>	Float, 8 wide, 2 decimals
<code>%10.4e</code>	Scientific
<code>%s</code>	String
<code>%d</code>	Integer
<code>%%</code>	Literal percent sign

0.9 Two MATLAB things with no Python equivalent

The semicolon suppresses output. A line without `;` prints its result to the Command Window. There's no `print()` needed — omitting the semicolon *is* printing. While debugging, leave semicolons off. In finished code, put them on.

Code sections. `%%` at the start of a line creates a runnable block:

Shortcut

Action

<code>Ctrl+Enter</code>	Run current section only
<code>Ctrl+Shift+Enter</code>	Run section, advance
<code>F5</code>	Run whole file
<code>F12</code>	Breakpoint
<code>Ctrl+I</code>	Auto-indent

This is like a Jupyter cell but inside a plain `.m` file. It's how you'll build this incrementally.

0.10 Functions live in files

Unlike Python, you can't define a function anywhere and call it. Two options:

1. **Own file** — `skew.m` must contain a function called `skew`
2. **End of a script** — all local functions go *after* all script code, and are only visible within that file

There's no `import`. If the file is in your Current Folder, MATLAB finds it.

0.11 Debugging

Python
MATLAB

import pdb; pdb.set_trace()
keyboard

python -m pdb on exception
dbstop if error

print(type(x))
class(x)

print(x.shape)
size(x)

locals()
whos

dbstop if error stops execution *at the failing line* with the workspace intact. Turn it on before your first real run; turn it off with dbclear all.

0.12 Errors you'll hit coming from Python

Message	What you did	Fix
Index exceeds matrix dimensions	Used 0-based indexing	Start at 1
Matrix dimensions must agree	* where you meant .*, or row vs column	Check size(), add . or (:)
Unable to perform assignment because the size of the left side is 3-by-1 and the size of the right side is 1-by-3	cross returned a row into a column slot	Force column with v(:)
Undefined function or variable	File not in Current Folder	Check the Current Folder panel
Too many output arguments	Called a 1-output function with [a,b] =	Check the function signature

PART 1 — Architecture

1.1 The hardcode boundary

This is the design decision that shapes everything.

Hardcoded (physical properties — no geometry can give you these):

Quantity	Value	Why it can't be derived
Total mass M	250 kg	You weigh it
Front axle mass Mf	114 kg	You weigh it
Rear axle mass Mr	136 kg	You weigh it
CG height h	325 mm	Measured by tilt test
Friction coefficient μ	0.72	Tyre property
Cornering speed v	12.5 m/s	Event assumption
Cornering radius R	3 m	Event assumption
Braking speed u	11.11 m/s	Event assumption
Braking distance s	4.1 m	Event assumption
Wheel mass m	5 kg	You weigh it
Unsprung mass/corner	15 kg	You weigh it
Spring rates	11.8 / 14.8 N/mm	Spring spec
Bump travel	25.4 mm	Design choice
Brake mount offset c	55 mm	CAD dimension, not a LOTUS hardpoint
Pinion diameter	22.5 mm	Component spec

Derived from hardpoints (everything else):

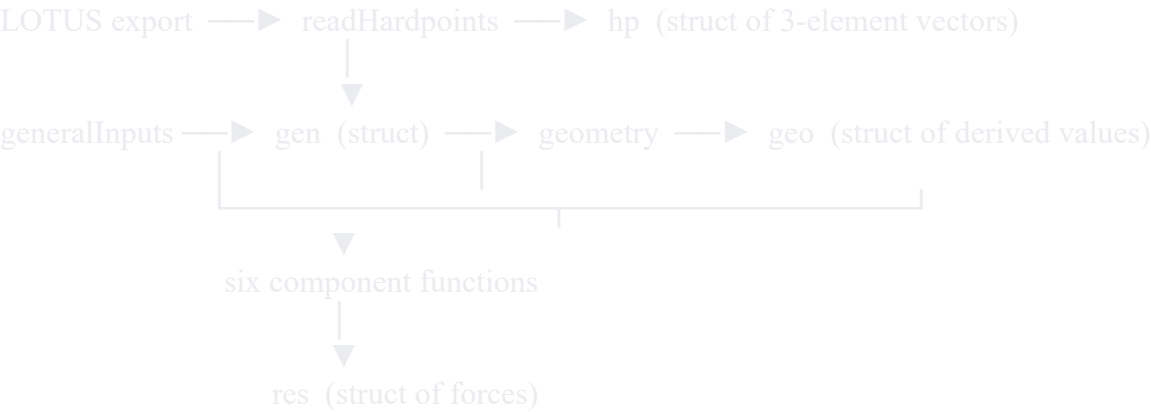
Wheelbase · track · loaded radius · ball joint heights · ball joint separation · hub-centre-to-ball-joint distances · kingpin inclination · caster · scrub radius · mechanical trail · steering arm length · tie rod length and direction · all eight A-arm leg lengths · A-arm lever arms · rocker arm lengths · rocker perpendicular distances · rocker ratios · pushrod angles and lengths · damper angle and length · all link unit vectors.

That's 15 hardcoded numbers versus roughly 60 derived ones. Change the car, change 15 numbers plus one hardpoint file.

1.2 Files

File	Type	Contains
readHardpoints.m	function	Loads the LOTUS export into a struct
geometry.m	function	Derives all ~60 geometric quantities
generalInputs.m	script	The 15 hardcoded physical values
frontKnuckle.m	function	Front upright forces
rearKnuckle.m	function	Rear upright forces
aArms.m	function	All four wishbones
rockers.m	function	Both rockers
hubs.m	function	Both hubs
rackPinion.m	function	Steering forces
report.m	script	Prints and exports everything
main.m	script	Driver

1.3 Data flow



Three structs total: hp (raw points), gen (hardcoded physical), geo (derived geometric). Every component function takes gen and geo and returns forces.

PART 2 — Getting hardpoints into MATLAB

2.1 About the .shk file

MATLAB cannot read LOTUS Shark's native .shk format — it's a proprietary binary. **Export from Shark instead.** You already have the right file: 6_0_LOTUS_SIMULATION.xlsx, which contains the full hardpoint table.

In Shark: File → Export or the report generator produces this. Keep using it.

2.2 Reading the spreadsheet

Command	Pattern	Returns
readmatrix	readmatrix(file, 'Range', 'B15:D30')	Numeric block as a matrix
readtable	readtable(file)	Table with named columns
readcell	readcell(file)	Everything as a cell array

Your front hardpoints sit in rows 15–30, columns B:D. Rear in rows 85–100. Both are 16×3 numeric blocks — readmatrix with a range is exactly right.

Procedure for readHardpoints.m:

1. Function takes the filename and the range string, returns a struct
2. Call readmatrix with the range → get a 16×3 matrix
3. Define a **cell array** of the 16 point names, in LOTUS's row order: LWBf, LWBr, LBJ, UWBf, UWBr, UBJ, PRw, PRr, TRo, TRi, Db, Dr, spindle, WC, RK1, RK2
4. Loop for i = 1:16, and inside, use dynamic field assignment: the field named by names{i} gets row i of the matrix
5. Return the struct

Python analogue: this is dict(zip(names, matrix)). MATLAB has no zip, so you loop.

2.3 The datum shift — do it immediately

Definition

Loaded tyre radius — the vertical distance from the wheel centre to the ground, with the tyre squashed under load. It is the moment arm for every force acting at the contact patch.

LOTUS stores hardpoints in its own coordinate frame where the ground is *not* at z = 0. You have to find where the ground actually is:



$$\text{ground_z} = \text{wheel_centre_z} - \text{loaded_radius}$$

Your values: front 247.89 – 234.95 = 12.94 mm, rear 247.84 – 234.95 = 12.89 mm.

Every height-derived quantity — ball joint heights, scrub radius, roll centre — must reference the same plane, or your moment arms are 13 mm out.

In MATLAB

1. Compute the offset from hp.WC(3) and the loaded radius
2. Get all field names with fieldnames
3. Loop over them, and for each, subtract the offset from element 3 of that field
4. Assign back with the dynamic-field syntax

Four lines inside the loop.

✔ Check

After shifting, print these four:

Point	Expected z
Front LBJ	132.11 mm
Front UBJ	311.91 mm
Rear LBJ	146.98 mm
Rear UBJ	322.93 mm

PART 3 — The geometry engine

This is the heart of the tool. Each subsection: what the quantity *is*, why it matters, the formula, the MATLAB commands, and your car's value to check against.

3.0 Three helper operations you'll use constantly

Write these as local functions at the bottom of `geometry.m`.

Unit vector. A vector divided by its own magnitude, so it has length 1 and carries only direction. $\rightarrow v / \text{norm}(v)$

Perpendicular distance from a point to a line. Given a point P, a point A on the line, and the line's unit direction e: $\rightarrow$ the vector $P - A$, minus its own projection onto e, then take norm. Projection is $\text{dot}(P-A, e) * e$.

Perpendicular distance from a point to a line, measured about an axis. Used for rockers and A-arms, where you want the moment arm about a rotation axis: $\rightarrow \text{abs}(\text{dot}(\text{cross}(\text{armVector}, \text{forceDirection}), \text{axisUnitVector}))$

That third one is the single most useful line in the whole project. It gives you the moment arm directly.

3.1 Wheelbase

Definition: the longitudinal distance between the front and rear axle centrelines. It sets how much vertical load shifts front-to-rear when you brake — a short wheelbase transfers more.

Formula: $L = x_{\text{rearWheelCentre}} - x_{\text{frontWheelCentre}}$

MATLAB: subtraction of two struct field elements.

Your value: 1626.00 mm

Report 9.0 states 1625 mm in the knuckle section and 1626 mm in the hub section. Derive it and the ambiguity disappears.

3.2 Track width

Definition: lateral distance between the tyre centrelines on one axle. It sets how much load shifts side-to-side in a corner — a narrow track transfers more.

Formula: $t = 2 * y_{\text{wheelCentre}}$ (for a symmetric car with the origin on the centreline)

MATLAB: multiply one field element by 2.

Your values: front 1200 mm, rear 1150 mm

3.3 Ball joint heights above ground

Definition — ball joint: a spherical bearing connecting the upright to a wishbone. It transmits force in any direction but no moment, which is why the wishbone behaves as a pin-jointed link.

Definition — the heights: the vertical distance from the ground plane up to each ball joint centre. These set the lever arms in the upright's moment balance.

Formula:



$a' = z_UBJ - z_ground$ (upper)
 $b' = z_LBJ - z_ground$ (lower)
 $H = a' - b'$ (ball joint separation)

Your values:

	a' (upper)	b' (lower)	H
Front	311.91 mm	132.11 mm	179.80 mm
Rear	322.93 mm	146.98 mm	175.95 mm

3.4 Ball joint force split

Definition: when a lateral force acts at the contact patch, the two ball joints react it as a **couple** — one pushes outboard, the other inboard. The split depends only on their heights.

Why it matters: it tells you which wishbone is more heavily loaded, and therefore which one to design most carefully.

Derivation (worth being able to reproduce in Design Event):



Force balance: $F_U + F_L + F_y = 0$
Moments about the contact patch: $F_U \cdot a' + F_L \cdot b' = 0$

Solving: $F_U = -F_y \cdot b' / (a' - b')$
 $F_L = +F_y \cdot a' / (a' - b')$

Your values:

	F_upper	F_lower
Front	$0.735 \times F_y$	$1.735 \times F_y$
Rear	$0.835 \times F_y$	$1.835 \times F_y$

The lower ball joint carries 2.4× the upper at the front. This is true for essentially every double wishbone with both joints above the ground.

3.5 Hub-centre to ball-joint distances (a and b)

Definition: vertical distances from the wheel centre up to the upper ball joint (a) and down to the lower ball joint (b). Report 9.0 uses these as the beam lengths in its upright moment balance.

Formula:



$$a = z_UBJ - z_wheelCentre$$
$$b = z_wheelCentre - z_LBJ$$

Your values:

	a (upper)	b (lower)	a + b
Front	76.96 mm	102.84 mm	179.80 mm
Rear	87.98 mm	87.97 mm	175.95 mm

⚠ **This does not match Report 9.0.** That report uses $a = b = 110.9$ mm, giving $a + b = 221.8$ mm — **23% larger** than your actual 179.80 mm, and symmetric where your front geometry is not.

Since $F1 = (f \cdot a - Tb) / (a + b)$, this changes the front knuckle ball-joint forces directly. Deriving from hardpoints is the correct answer; the 110.9 figure appears to come from a different revision or a different measurement convention.

Note the rear *is* nearly symmetric (87.98 vs 87.97) — so whoever measured it was working with a symmetric upright, just not at 110.9 mm.

3.6 Steering axis (kingpin axis)

Definition: the imaginary line through both ball joints. The upright rotates about it when you steer. Everything about steering feel — self-centring, kickback, effort — comes from this line's position.

Formula: $\hat{u}_{KP} = (p_UBJ - p_LBJ) / |p_UBJ - p_LBJ|$

MATLAB: subtract, divide by norm.

3.7 Kingpin inclination and caster angle

Definition — kingpin inclination (KPI): the steering axis's lean when viewed from the front of the car. Leaning the top inboard reduces scrub radius and makes the car self-centre by lifting itself slightly as you steer.

Definition — caster: the steering axis's lean when viewed from the side. Positive caster (top leaning rearward) puts the contact patch behind the steering axis, so the wheel trails like a shopping-trolley castor and self-centres.

Formula:



$$KPI = \text{atan2d}(-\hat{u}_{KP,y}, \hat{u}_{KP,z})$$
$$Caster = \text{atan2d}(-\hat{u}_{KP,x}, \hat{u}_{KP,z})$$

MATLAB: use `atan2d` — the two-argument arctangent that gets the quadrant right, unlike `atand`.

Your values: front KPI **3.61°**, caster **2.60°**. Rear KPI **2.32°**, caster **1.46°**.

These match the LOTUS static output exactly, which confirms your unit vector and datum are both correct.
Good early checkpoint.

3.8 Steering axis ground intercept

Definition: where the steering axis pierces the ground plane. Scrub radius and mechanical trail are both measured from this point.

Formula: walk along the axis from the lower ball joint until z reaches zero:



$$t = -z_{LBJ} / \hat{u}_{KP,z}$$
$$p_{SA0} = p_{LBJ} + t \cdot \hat{u}_{KP}$$

MATLAB: scalar `t`, then a vector add of `t` times the unit vector.

3.9 Scrub radius

Definition: the lateral distance at ground level between the steering-axis intercept and the tyre centreline.

Why it matters: it is the lever arm through which braking force twists the steering. Large scrub radius means a wheel locking or hitting a bump yanks the steering wheel. It also sets how hard the car is to steer at rest.

Formula: $r_s = y_{SA0} - y_{contactPatch}$

Your values: front **63.78 mm** (magnitude), rear 79.92 mm.

Report 9.0 states 63.77 mm. **Your derived value matches to 0.01 mm.** Strong evidence the hardpoints and the report describe the same car.

The sign comes out negative in the raw calculation, which just means the axis intercept is inboard of the tyre centreline. Take `abs()` when using it as a lever arm.

3.10 Mechanical trail

Definition: the longitudinal distance between the steering-axis ground intercept and the contact patch. Positive trail means the contact patch is *behind* the axis, so cornering force naturally straightens the wheel.

Why it matters: it's the main source of steering feel. Too little and the steering goes numb; too much and it's heavy.

Formula: $t_m = x_{contactPatch} - x_{SA0}$

Your values: front **18.00 mm**, rear -1.02 mm.

3.11 Steering arm length

Definition: the perpendicular distance from the steering axis to the tie rod's line of action. It is the lever the tie rod pulls on to rotate the upright.

Why it matters: it converts steering-axis moment into tie rod force. Halve it and you double the tie rod load.

Formula:



$$\hat{u}_{TR} = (p_{TRinboard} - p_{TRoutboard}) / |\dots|$$
$$L_s = | [(p_{TRoutboard} - p_{LBJ}) \times \hat{u}_{TR}] \cdot \hat{u}_{KP} |$$

MATLAB: abs(dot(cross(armVector, u_TR), u_KP)).

Your values: front **45.05 mm**, rear (toe link) 35.67 mm.

Report 9.0 states 45 mm. **Derived value matches to 0.05 mm.** This settles a question that was genuinely open — an earlier team document assumed 100 mm, which would have halved the tie rod force.

3.12 A-arm leg lengths

Definition: a wishbone (A-arm) has two legs meeting at the ball joint. Each leg is a two-force member carrying pure axial load — tension or compression.

Formula: distance from the ball joint to each inboard pivot.

Your values:

Arm	Fore leg	Aft leg
Front upper	305.81 mm	343.11 mm
Front lower	315.77 mm	391.91 mm
Rear upper	367.98 mm	269.58 mm
Rear lower	396.47 mm	266.65 mm

These match your 2020 drawing set to 0.03 mm across all eight. Your hardpoints and your CAD are the same car — worth stating in the report.

3.13 A-arm lever arms (for the pushrod balance)

Definition — inboard pivot axis: the line joining a wishbone's two chassis pivots. The wishbone rotates about it as the suspension moves.

Definition — lever arms: perpendicular distances from that axis to (i) the pushrod pickup and (ii) the ball joint. Their ratio converts wheel force into pushrod force.

Why it matters: this is the first stage of the motion ratio chain. The pushrod picks up inboard of the ball joint, so it sees *more* force than the wheel.

Formula:



$\hat{e} = (\mathbf{p_pivot2} - \mathbf{p_pivot1}) / |\dots|$
 d_PR = perpendicular distance from $\hat{e}$ -axis to the pushrod pickup
 d_BJ = perpendicular distance from $\hat{e}$ -axis to the ball joint

MATLAB: use the projection form — $\mathbf{v}$ minus $\text{dot}(\mathbf{v}, \hat{e}) \cdot \hat{e}$, then norm.

Your values:

	Pivot→pushrod	Pivot→ball joint	Ratio
Front (lower arm)	254.88 mm	309.10 mm	0.825
Rear (upper arm)	211.07 mm	268.89 mm	0.785

⚠ **Which arm carries the pushrod is asymmetric on your car.** Front pushrod attaches to the **lower** wishbone; rear attaches to the **upper**. I verified this by computing the perpendicular distance from the pushrod pickup to each wishbone's plane — front is 22.18 mm from the lower plane vs 144.43 from the upper; rear is 18.27 from the upper vs 187.44 from the lower. Unambiguous.

Report 9.0's hub section uses lever arms of 245.13 and 408.55 mm. The pushrod arm is close to your derived 254.88; the ball-joint arm is 32% off. Note also that $245.13 = 0.600 \times 408.55$ *exactly* — those numbers were constructed, not measured.

3.14 Rocker geometry

Definition — rocker (bellcrank): a rigid triangular lever with three holes. It pivots on the chassis and converts pushrod motion into damper motion. It exists so the damper can be packaged anywhere and so you can tune the motion ratio.

Definition — arm length L : straight-line distance from the pivot hole to the pushrod or damper hole.

Definition — perpendicular distance d : distance from the pivot **axis** to the force's **line of action**. This is what actually appears in the moment balance. Related by $d = L \cdot \sin \theta$, where θ is the angle between the arm and the force direction.

Why the distinction matters: if a force pulled straight along its arm, it would produce no moment at all. Only the perpendicular component does work.

Formula:



$\hat{e}_pivot = (\mathbf{p_RK2} - \mathbf{p_RK1}) / |\dots|$
 $\hat{u}_PR$ = direction from rocker pushrod hole toward the A-arm pickup
 $\hat{u}_D$ = direction from rocker damper hole toward the chassis mount

$d_PR = |(\mathbf{p_pushrodHole} - \mathbf{p_pivot}) \times \hat{u}_PR \cdot \hat{e}_pivot|$
 $d_D = |(\mathbf{p_damperHole} - \mathbf{p_pivot}) \times \hat{u}_D \cdot \hat{e}_pivot|$

Your values:

	Arm→pushrod	Arm→damper	d_PR	d_D	θ_PR	θ_D	Ratio d_PR/d_D
Front	145.91 mm	137.28 mm	131.40	127.15	64.23°	67.85°	1.033
Rear	86.05 mm	53.79 mm	82.86	53.78	74.35°	88.87°	1.541

Two things worth knowing. First, the arm lengths reproduce the drawing dimensions exactly. Second, **Report 9.0 assigns them the opposite way round** — it uses 137.28 as the pushrod arm at the front and 54.11 at the rear, when the hardpoints say 145.91 and 86.05. That inverts the ratio: the report concludes the damper sees *less* force than the pushrod, when the geometry says it sees **more** (1.03× front, 1.54× rear).

Note the rear damper sits at 88.87° to its arm — essentially perpendicular. That's good rocker design, and it means $d \approx L$ there.

3.15 Pushrod and damper directions

Definition — pushrod: a slender strut carrying wheel load from the wishbone up to the rocker. Always in compression in bump (if it were in tension it would be a pullrod).

Definition — angle from vertical: the tilt of the strut's axis. Force scales roughly as $1/\cos \theta$, so a steeply inclined pushrod sees more load.

Formula: $\theta = \arccos(|\hat{u}_z|)$

Your values:

	Length	Angle from vertical
Front pushrod	200.74 mm	29.75°
Rear pushrod	207.84 mm	39.85°
Front damper	196.39 mm	45.79°
Rear damper	201.68 mm	75.00°

The front damper's 45.79° is close to Report 9.0's "coil spring angle 47°" — so that input was real. The pushrod angles it assumes (5° and 35°) match nothing in the geometry.

3.16 Link unit vectors

Definition: for each suspension link, the unit vector pointing from its outboard (upright) end to its inboard (chassis) end. Positive tension in the link pulls the upright toward the chassis along this direction.

Why you need them: they are the columns of the equilibrium matrix if you later build the full 3-D link solve.

MATLAB procedure:

1. Build a **cell array** of six triplets: {label, outboardFieldName, inboardFieldName}
2. Preallocate a 6×3 matrix with zeros
3. Loop for $i = 1:6$, extract the two names with curly braces, look up the points with dynamic field access, subtract, normalise
4. Store into row i

 **Check — front upper fore leg must come out as:**



[-0.1515 -0.9877 -0.0387]

Match to four decimals and your file reading, datum shift and normalisation are all correct simultaneously.

PART 4 — The general inputs block

The only hand-typed numbers. Put them in `generalInputs.m` as a struct, one field per line, with a unit comment.



gen.M	250	kg	total vehicle mass with driver
gen.Mf	114	kg	mass on the front axle
gen.Mr	136	kg	mass on the rear axle
gen.g	9.81	m/s ²	
gen.h	0.325	m	CG height above ground
gen.mu	0.72	-	tyre-road friction coefficient
gen.v	12.5	m/s	cornering speed
gen.R	3.0	m	cornering radius
gen.u	11.11	m/s	speed at brake application
gen.S	4.1	m	braking distance to a stop
gen.mWheel	5	kg	mass of one wheel assembly
gen.mUns	15	kg	unsprung mass per corner
gen.kF	11.8	N/mm	front spring rate
gen.kR	14.8	N/mm	rear spring rate
gen.bump	25.4	mm	bump travel
gen.c	0.055	m	brake mount to rear upright centre
gen.dPin	22.5	mm	pinion pitch diameter
gen.accBrk	11.77	m/s ²	braking deceleration (1.2 g)
gen.accCor	13.73	m/s ²	cornering acceleration (1.4 g)

Unit discipline. Pick one system and comment every line. My recommendation: **millimetres and newtons for geometry, metres for anything entering a torque**. Convert at the point of use, not in storage. Mixing metres and millimetres never raises an error — you just get answers wrong by 1000.

PART 5 — The force calculations

Each component: what's being computed, why, the formula chain, and what to watch.

5.1 Front upright — braking case

What an upright is

The **upright** (also called the knuckle) is the structural part that holds the wheel bearing. It connects to both wishbones through ball joints, to the tie rod through a steering arm, and carries the brake caliper. Every force the tyre generates passes through it.

Definitions

Quantity	Definition
Static wheel load F_w	Vertical force on one wheel with the car standing still.
Longitudinal load transfer	Vertical load that shifts from the rear axle to the front when braking, because the inertia force acts at CG height while the reaction is at ground level.
Total vertical load F_v	Static load plus transfer — what the tyre actually carries while braking.
Frictional force f	The braking force at the contact patch, limited by friction: $\mu \times F_v$.
Braking torque T_b	The moment the brake applies about the wheel axis, $f \times \text{loaded radius}$. Reacted by the caliper, which bolts to the upright.
F_1, F_2	Reaction forces at the upper and lower ball joints.

Chain



$$F_w = M_f \cdot g / 2$$

$$\Delta F = M \cdot a_{\text{brake}} \cdot h / (2 \cdot L)$$

$$F_v = F_w + \Delta F$$

$$f = \mu \cdot F_v$$

$$T_b = f \cdot R_{\text{loaded}}$$

$$F_1 = (f \cdot b - T_b) / (a + b)$$

$$F_2 = f - F_1$$

Unit trap: T_b is a torque, so R_{loaded} must be in **metres**. Then $f \cdot b$ in the F_1 numerator must also be in metres. Convert a , b and R_{loaded} from mm to m at the top of the function — otherwise F_1 is out by 1000.

a and b come from §3.5 — derived, not typed. Your values differ from Report 9.0's, so F_1 and F_2 will too. That's correct behaviour.

5.2 Front upright — cornering case

Definitions

Quantity	Definition
Centrifugal couple F_c	Extra vertical load pushed onto the outer wheel as the car's inertia tries to roll it outward in a corner.
Gyroscopic effect F_g	A spinning wheel resists having its axis tilted. Turning the car precesses each wheel, producing a small additional load.
Net vertical load F_{net}	Static + centrifugal + gyroscopic.
Lateral force F_L	Sideways force at the contact patch: $\mu \times F_{\text{net}}$.
F_3, F_4	Ball joint reactions to the lateral force.

Chain



$$F_c = M \cdot v^2 \cdot h / (2 \cdot R \cdot t_{front})$$
$$F_g = 4 \cdot m_{wheel} \cdot v^2 \cdot R_{loaded} / (2 \cdot R)$$
$$F_{net} = F_w + F_c + F_g$$
$$F_L = \mu \cdot F_{net}$$
$$F_3 = F_L \cdot (R_{loaded} - b) / (a + b)$$
$$F_4 = F_L + F_2$$

t_front and R_loaded come from geometry; a and b from §3.5.

The gyroscopic term is small (around 5% of the centrifugal term). It's physically real but Report 9.0's hub section inflates it hugely — worth computing properly here.

5.3 Steering arm force

Definitions

Quantity	Definition
Steering-axis torque T_s	The moment trying to rotate the upright about the steering axis, generated by contact-patch force acting through the scrub radius.
Tie rod force F_5	The force in the link from the rack to the steering arm, resisting that torque.

Chain



$$f_o = \mu \cdot F_w$$
$$T_s = f_o \cdot r_{scrub}$$
$$F_5 = T_s / L_s$$

Both r_scrub (§3.9) and L_s (§3.11) are derived. Your derived values match Report 9.0's to 0.01 mm and 0.05 mm respectively, so F5 will reproduce.

5.4 Rear upright

Same structure, no steering. Add drive torque.



$$W_r = M_r \cdot g / 2$$
$$F_v = W_r + \Delta F$$
$$F_c = \mu \cdot F_{z, rearOuter} \quad \leftarrow \text{derived, not the report's } 3 \cdot F_v$$
$$F_b = \mu \cdot F_v \quad \leftarrow \text{derived, not the report's } 1.5 \cdot W_r$$
$$T_b = F_b \cdot c$$
$$F_t = \mu \cdot F_v$$

Report 9.0 uses $F_c = 3 \cdot F_v$ and $F_b = 1.5 \cdot W_r$ — unexplained multipliers with no stated basis. Compute from μ instead and note the change.

5.5 Wishbones

Definitions

Quantity	Definition
Wishbone load w	Share of axle weight carried by one wishbone.
$F1$	Load added by longitudinal transfer under braking.
$F2$	Load added by lateral transfer in a corner.

Chain



$W = M_{axle} \cdot g / 4$
 $F1 = M \cdot a_{brake} \cdot h / (4 \cdot L)$
 $F2 = M \cdot a_{corner} \cdot h / (4 \cdot t_{axle})$

Write **one function** taking axle mass and track width, call it four times. L and t_{axle} are derived.

5.6 Rockers

Definitions

Quantity	Definition
Wheel rate K_w	Force per millimetre of <i>vertical wheel</i> travel. Distinct from spring rate, which is measured at the spring.
Motion ratio MR	Geometric leverage between wheel and damper. Your LOTUS export reports both the ratio and the travels, so the convention is directly checkable: 10 mm wheel produces 7.00 mm damper , ratio 1.428 — that's wheel ÷ damper .
Bump force	Vertical wheel load at full suspension compression: $K_w \times \text{bump travel}$.
Pivot reaction	Force the rocker pushes into its chassis mount — the vector sum of pushrod and damper forces.

Chain



$K_w = k_{spring} / MR^2$ ← wheel÷damper convention
 $F_{bump} = K_w \cdot \text{bump_travel}$
 $F_{PR} = F_{bump} \cdot (d_{BJ} / d_{PR_arm}) / \cos(\theta_{pushrod})$ §3.13, §3.15
 $F_{damper} = F_{PR} \cdot d_{PR} / d_D$ §3.14
 $R_{pivot} = -(F_{PR} \cdot \hat{u}_{PR} + F_{damper} \cdot \hat{u}_D)$

Every geometric term derived. Nothing typed.

The check that validates the whole chain



$F_{\text{damper}} \approx F_{\text{wheel}} \times MR$ within 10%

Motion ratio is wheel travel ÷ damper travel, and by virtual work force scales the same way. If this fails, either your MR or your rocker hardpoints are wrong. One line, catches an entire class of error.

5.7 Hubs

Definitions

Quantity	Definition
Hub	The rotating part that carries the wheel studs and the brake rotor, running on bearings around the upright.
Deceleration	Derived from the braking event: $u^2/(2S)$.
Braking torque	frictional force × loaded radius — reacted between the rotor bolt circle and the wheel studs.
Bump force	Vertical load at full compression, from the rocker chain.

Chain



$decel = u^2 / (2 \cdot S)$
 $F_v = static + M \cdot decel \cdot h / (2 \cdot L)$
 $f = \mu \cdot F_v$
 $T_b = f \cdot R_{\text{loaded}}$
 $F_c = M \cdot u^2 \cdot h / (2 \cdot R \cdot t_{\text{front}})$
 $F_{\text{bump}} = \text{from §5.6}$

R_{loaded} , L , t_{front} all derived. This is where deriving pays off most — Report 9.0's hub section uses a tyre radius of 311 mm against 234.95 from your hardpoints, a **32% error** in every torque.

5.8 Rack and pinion

Definitions

Quantity	Definition
Rack and pinion	Converts steering wheel rotation into sideways rack travel, which pushes the tie rods.
M_v	Steering-axis moment from vertical wheel load acting through the scrub radius, modified by kingpin and steer angle.
M_l	Steering-axis moment from lateral force acting through caster trail.
M_t	Steering-axis moment from tractive/braking force difference across the axle, acting through scrub radius.
Rack force	Sum of both tie rod forces resolved along the rack axis.
Pinion torque	Rack force × pinion pitch radius.

Chain



$R_{fl} = F_w + \text{lateral transfer} \quad (\text{loaded side})$
 $R_{fr} = F_w - \text{lateral transfer} \quad (\text{unloaded side})$

$M_v = -(R_{fl} + R_{fr}) \cdot (r_s \cdot \sin(\lambda) \cdot \sin(\delta)) + (R_{fl} - R_{fr}) \cdot (r_s \cdot \sin(v) \cdot \cos(\delta))$
 $M_l = -(R_{fl} + R_{fr}) \cdot (R_{loaded} \cdot \tan(v))$
 $M_t = (R_{fl} - R_{fr}) \cdot \mu \cdot r_s$

$M = |M_v| + |M_l| + |M_t|$

$F_{tierod} = M / L_s$
 $T_{pinion} = F_{rack} \cdot d_{pinion} / 2$

$r_s, \lambda, v, L_s, R_{loaded}$ all derived (§3.7, §3.9, §3.11).

Take `abs()` before summing. M_l computes negative but Report 9.0 adds its magnitude. Without `abs()` you'll get 3145 instead of 38239.

Steering wheel torque sanity check: T_{pinion} should land between **15 and 25 N·m**. Above that the steering is too heavy — a design finding, not just a number.

PART 6 — Validation

6.1 Geometric checks — run these first

These verify your hardpoint reading and derivation before any force is computed.

Quantity	Expected
Wheelbase	1626.00 mm
Front / rear track	1200 / 1150 mm
Front LBJ height above ground	132.11 mm
Front UBJ height above ground	311.91 mm
Front kingpin inclination	3.61°
Front caster	2.60°
Front scrub radius	63.78 mm
Front steering arm	45.05 mm
Front upper fore leg	305.81 mm
Front upper fore unit vector	[−0.1515, −0.9877, −0.0387]

The kingpin and caster angles are the strongest check — they come out of your derivation and appear independently in the LOTUS static output. Matching both to two decimals means your datum, your unit vectors and your angle conventions are all correct.

6.2 Where you'll match Report 9.0 and where you won't

Quantity	Derived vs report
Scrub radius	63.78 vs 63.77 — matches
Steering arm	45.05 vs 45 — matches
A-arm legs	match drawings to 0.03 mm
Track, wheelbase	match
a, b	76.96 / 102.84 vs 110.9 / 110.9 — differs
Rocker arm assignment swapped	
Hub tyre radius	234.95 vs 311 — differs
Hub lever arms	254.88 / 309.10 vs 245.13 / 408.55 — differs

Where the geometry matches, your forces will reproduce the report. Where it doesn't, they won't — and the derived value is the right one.

Print both in your output table with a percentage difference column. That turns a disagreement into a documented finding rather than an unexplained discrepancy, which is exactly what Design Event judges want to see.

6.3 Coding checks

Write these as assert calls so the script stops rather than producing plausible rubbish.



- ☐ Every unit vector has norm 1 tol 1e-12
- ☐ Kingpin and caster match LOTUS tol 0.01°
- ☐ Ball joint ratio = $a'/(a'-b')$ and $b'/(a'-b')$
- ☐ $F_{damper} \approx F_{wheel} \times MR$ tol 10%
- ☐ Steering wheel torque 15–25 N·m
- ☐ No wishbone leg force above 6 kN

Assert syntax: `assert(condition, 'message with %f', value)`. The message uses `fprintf` specifiers, so when it fails you see the number that broke it.

PART 7 — Run order



1. Set format long g, create the folder
2. Write readHardpoints.m → print all 16 points, eyeball against LOTUS
3. Apply the datum shift → front LBJ $z = 132.11$
4. Write the three helper functions → test each on a hand-solvable case
5. Write geometry.m section by section, checking each against §6.1
6. Verify kingpin = 3.61° , caster = 2.60° ← the key checkpoint
7. Write generalInputs.m → 19 numbers, all commented
8. frontKnuckle.m → braking, cornering, steering blocks
9. rearKnuckle.m
10. aArms.m → one function, four calls
11. rockers.m → check $F_{\text{damper}} \approx F_{\text{wheel}} \times MR$
12. hubs.m
13. rackPinion.m → abs() before summing moments
14. report.m → derived vs report comparison table
15. Export to results.xlsx

Step 6 is the one to be patient about. If kingpin and caster both come out right, everything upstream is verified — file reading, datum, subtraction order, unit vectors, and your angle convention. Every later stage builds on those.

One habit from Python that will serve you here

You're used to running a cell, looking at the output, running the next. MATLAB's %% sections give you exactly that — and it's the right way to build this.

Write one derivation, run the section with Ctrl+Enter, print the result, check it against the table in Part 3, move on. Don't write geometry.m end-to-end and then run it. Sixty derived quantities means sixty places for a sign error, and finding them one at a time takes minutes where finding them all at once takes hours.

When you get stuck

Post the error and which section you're on. If it runs but a number looks wrong, post the value — the tables in Part 3 are specific enough that I can usually tell you which line is at fault.